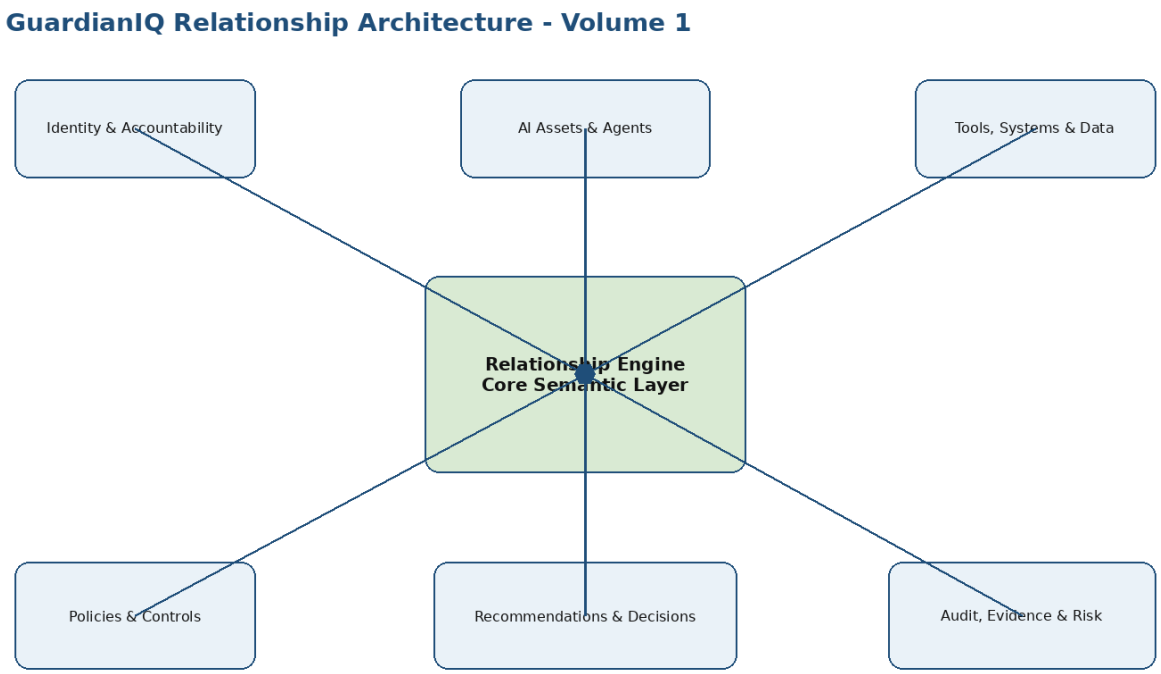
GuardianIQ Engineering Handbook

Volume 1: Relationship Architecture & Enterprise Implementation Guide

Version 1.0 | Confidential Working Document | Innovant.ai



Document Control

Field	Value
Platform	GuardianIQ
Handbook Volume	Volume 1
Title	Relationship Architecture & Enterprise Implementation Guide
Primary Audience	Product Owners, Architects, Backend Developers, Frontend Developers, QA Engineers, Project Manager
Primary Purpose	Define the canonical relationship architecture and enterprise implementation approach for GuardianIQ.
Pre-requisites	Phase 0 Identity/Foundation, Phase 1 Registry, Phase 2 Governed Workflow Scheduling and AI Agent Execution.
Primary Source	GuardianIQ Step 3 - Relationship Architecture Foundation.
Status	Implementation Reference Draft v1.0

How to use this handbook: This volume is the canonical reference for relationship semantics, relationship lifecycle, relationship validation, governance graph traversal, and implementation principles. Later volumes should reference this handbook rather than redefining relationship meanings.

Section	Title
A	Executive Context and Product Philosophy
B	Relationship Architecture Principles
C	Canonical Relationship Ontology
D	Enterprise Relationship Taxonomy
E	Governable Object Pattern
F	Generic Relationship Pattern
G	Specific vs Generic Relationship Strategy
H	Relationship Lifecycle and State Management
I	Relationship Engine Architecture
J	Relationship Resolution and Graph Traversal
K	Validation Engine and Rule Catalogue
L	Security, RBAC, ABAC and Accountability
M	Audit, Provenance and Evidence Model
N	Logical Data Model Guidance
O	API Architecture and Contract Patterns
P	Backend Service Implementation Guide
Q	Frontend and UI/UX Reference
R	Performance, Caching and Scaling
S	Examples and Scenarios
T	Implementation Roadmap, QA and Definition of Done
U	Appendices

Section A - Executive Context and Product Philosophy

GuardianIQ is being designed as an enterprise AI governance and accountability platform. Earlier phases established the identity foundation, registry foundation, and governed workflow execution layer. Volume 1 defines the relationship architecture that allows all those capabilities to behave as one coherent platform rather than isolated modules.

The relationship layer is the semantic backbone of GuardianIQ. It explains who owns what, which AI asset used which tool, which workflow produced which recommendation, which policy governed it, who approved it, what evidence supported it, what action resulted, and what audit event proves the lifecycle later.

Without a disciplined relationship architecture, GuardianIQ would become a set of screens and tables. With a disciplined relationship architecture, it becomes a governance graph that can support explainability, impact analysis, policy intelligence, AI agent control, audit readiness and future enterprise graph analytics.

Core design statement: Every governable object in GuardianIQ must be traceable to ownership, policy, evidence, action and audit. Every AI-driven decision should be reconstructable as a relationship chain.

Previous Phase	Relationship Dependency Introduced	Why It Matters in Volume 1
Phase 0 - Identity, Access & Accountability	Users, roles, departments, approval groups, delegations, audit base	Defines actors, authority, ownership and accountability.
Phase 1 - Governance Registry	AI models, AI agents, tools, workflows, data sources, applications	Defines governable assets and operational context.
Phase 2 - Governed Workflow Scheduling	Schedules, runs, outputs, failures, agent boundaries	Defines runtime relationships and execution traceability.
Phase 3 - Relationship Architecture	Generic and specific relationships, graph semantics, validation and traversal	Defines the common relationship layer all future capabilities depend on.

External Reference Alignment

This handbook is an enterprise product architecture document, not a compliance manual. It uses external standards as directional alignment for accountability, provenance, governance, lifecycle risk management and traceability. Reference documents include NIST AI RMF, ISO/IEC 42001, EU AI Act direction and W3C PROV-O concepts. The implementation must remain framework-neutral so GuardianIQ can later map evidence to multiple regulatory and governance frameworks.

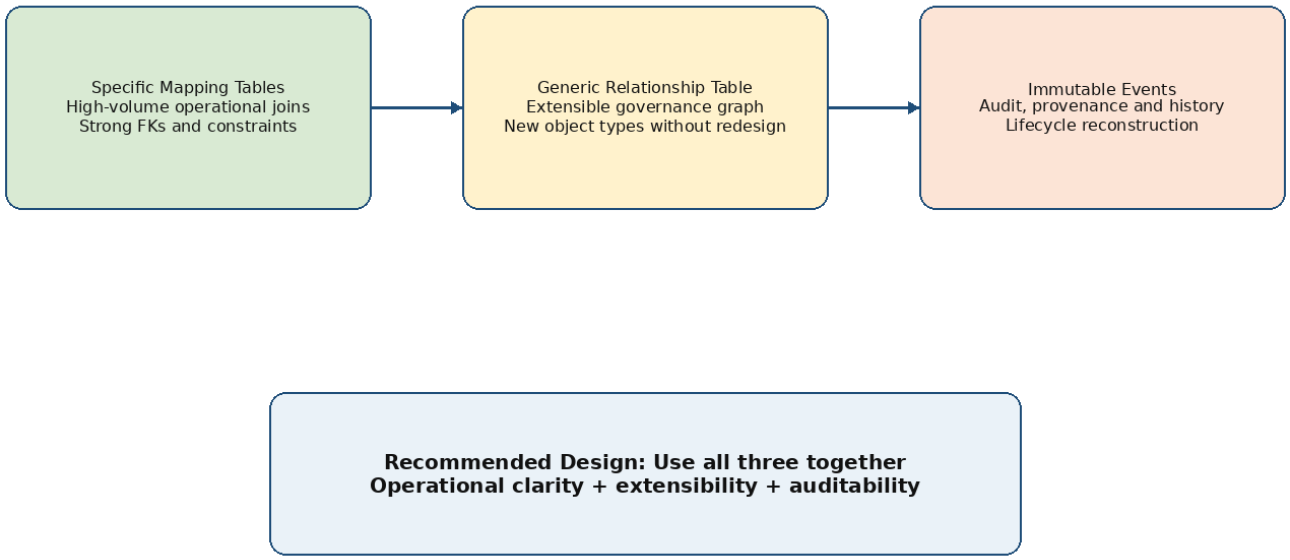
Reference	Relevant Concept	GuardianIQ Design Implication
NIST AI RMF	Govern, map, measure and manage AI risk across lifecycle	Relationships must connect AI assets, context, controls, decisions and outcomes.
ISO/IEC 42001	AI management system requirements and continual improvement	Relationships must connect ownership, policies, reviews, monitoring and improvement actions.
EU AI Act direction	Risk-based oversight, documentation, human oversight and logging	Relationships must preserve risk classification, human decisions and traceability.
W3C PROV-O	Provenance across entities, activities and agents	GuardianIQ should model actor, activity, evidence and outcome relationships.

Section B - Relationship Architecture Principles

ID	Principle	Meaning
P1	Accountability First	Every governable object must resolve to an accountable owner or responsibility assignment.
P2	Relationships Are Time-Aware	Ownership, permissions, policy bindings and delegations change over time; effective dates are mandatory for governed relationships.
P3	Current State Is Not History	Operational tables hold current state; governance events preserve history and audit reconstruction.
P4	Humans and Machines Are Actors	Users, groups, service accounts, AI agents and system workers can initiate, own, review, execute or trigger events.
P5	Policy Governs Relationships	Policies do not govern only objects; policies govern an object in a context, relationship and operation.
P6	Evidence Supports Trust	Every high-risk decision, approval, violation and action should link to evidence or explanation.
P7	Hybrid Relationship Model	Use specific tables for high-volume controlled joins, generic relationships for graph extensibility, and events for history.
P8	No Orphan Governance Objects	AI models, agents, workflows, tools, data sources, policies and schedules must not be activated without ownership.

P9	No Silent Relationship Changes	Create, update, revoke, suspend or expire relationship changes must generate governance events.
P10	Relationships Drive UI and APIs	APIs should return relationship-aware objects; UI should show connected governance context, not flat records only.

Hybrid Relationship Model



Section C - Canonical Relationship Ontology

The ontology defines the objects and semantic relationships that GuardianIQ must understand consistently across modules. The implementation may use multiple tables and APIs, but the meaning of relationships must not change from module to module.

Ontology Element	Definition	Example
Actor	Human, group, machine or service identity that performs or owns a governance activity.	User, AI Agent, Approval Group, Scheduler Worker
Governable Object	Any object under governance control.	AI Model, Workflow, Policy, Recommendation, Tool, Data Source, Action
Relationship	A semantic connection between two governable objects or actors.	AI_AGENT USES_TOOL TOOL-ERP
Responsibility	A relationship assigning accountability for an object.	User is OWNER of AI Model
Policy Binding	A relationship connecting a policy to a governed object/context.	Policy governs workflow
Evidence Link	A relationship connecting an artifact to decision support.	Evidence supports approval
Event Reference	Immutable record that references actors and objects.	WORKFLOW_RUN_COMPLETED references run
Scope	The boundary where a relationship is valid.	Workflow, BU, department, region, risk level
Lifecycle State	The state of relationship validity.	ACTIVE, SUSPENDED, REVOKED
Temporal Window	Effective time range for a relationship.	2026-06-01 to 2026-12-31

Canonical relationship record example

```
{
  "relationship_id": "REL-2042",
  "tenant_id": "TEN-INNOVANT",
  "source_type": "AI_AGENT",
  "source_id": "AGT-GOV-REVIEW-001",
  "relationship_type": "USES_TOOL",
  "target_type": "TOOL",
  "target_id": "TOOL-REGISTRY-READ",
  "relationship_scope": "WORKFLOW:WF-AI-MODEL-RISK-REVIEW",
  "effective_from": "2026-06-15T00:00:00+05:30",
  "effective_to": null,
  "status": "ACTIVE",
  "metadata_json": {
    "access_mode": "READ_ONLY",
    "approval_required_for_execute": true
  }
}
```


Section D - Enterprise Relationship Taxonomy

Relationship Type	Category	Source	Target	Implementation Meaning
OWNED_BY	Accountability	Any Governable Object	Actor / Department / Group	Mandatory for activation where object is high-risk.
REVIEWED_BY	Governance Review	Recommendation / Policy / Evidence	Actor / Group	Captures review responsibility.
APPROVED_BY	Formal Approval	Approval Request / Policy / Schedule	Actor / Group	Must preserve delegated approvals.
EXECUTED_BY	Execution	Action / Workflow Run	Actor / Agent / System	Identifies executor.
USES_MODEL	AI Asset Usage	AI Agent / Workflow	AI Model	Controls model lineage.
USES_TOOL	Tool Permission	AI Agent / Workflow	Tool	Controls read/write/execute boundary.
USES_DATA_SOURCE	Data Access	Workflow / Tool / Agent	Data Source	Supports data sensitivity governance.
PARTICIPATES_IN_WORKFLOW	Workflow Participation	Agent / Model / Tool	Workflow	Operational context.
GOVERNED_BY	Policy Binding	Any Governable Object	Policy	Policy applicability relationship.
EVALUATED_BY	Policy Evaluation	Recommendation / Action	Policy Evaluation	Dynamic evaluation result.
SUPPORTED_BY	Evidence	Recommendation / Approval / Decision	Evidence	Explainability and traceability.
RESULTS_IN	Outcome	Recommendation / Decision	Action / Outcome	Decision-to-action chain.
TRIGGERS	Event Flow	Object / Policy / Schedule	Event / Notification	Event chain.
ESCALATED_TO	Escalation	Approval / Violation / Failure	Actor / Group	Escalation accountability.
DELEGATED_TO	Delegation	Actor	Actor	Temporary authority transfer.
BELONGS_TO	Organization	User / Department	Department / Business Unit	Org hierarchy.
MEMBER_OF	Group Membership	User	Approval / Reviewer Group	Group routing.
HAS_PERMISSION	RBAC	Role / Actor	Permission	Authorization baseline.
VIOLATES	Compliance	Agent / Action / Run	Policy / Control	Violation recording.
REMEDIATED_BY	Remediation	Violation	Action	Governance improvement.

Section E - Governable Object Pattern

GuardianIQ should not force every object into one physical table. However, every governable object must expose a consistent set of governance fields so relationship, audit and policy services can treat objects uniformly.

Field	Mandatory?	Purpose	Example
tenant_id	Yes	Client / organization boundary	TEN-INNOVANT
object_type	Yes	Canonical object type	AI_AGENT
object_id	Yes	Stable identifier	AGT-0001
object_code	Recommended	Human-readable code	GOV_REVIEW_AGENT_001
object_version	Conditional	Versioned reproducibility	v1.3
status	Yes	Lifecycle status	ACTIVE
owner_user_id / owner_group_id	Yes for governed objects	Accountability	USR-1021
risk_level	Recommended	Governance risk	HIGH
created_at / updated_at	Yes	Operational timestamps	2026-06-15T09:00:00+05:30
metadata_json	Yes	Flexible AI/context metadata	{"execution_mode":"RECOMMEND_ONLY"}

Governable object reference payload

```
{
  "tenant_id": "TEN-INNOVANT",
  "object_type": "WORKFLOW",
  "object_id": "WF-AI-MODEL-RISK-REVIEW",
  "status": "ACTIVE",
  "owner_user_id": "USR-AI-GOV-ADMIN",
  "risk_level": "HIGH",
  "metadata_json": {
    "business_unit": "Global",
    "criticality": "High",
    "requires_approval_group": "AI Governance Board"
  }
}
```

Section F - Generic Relationship Pattern

The generic relationship pattern is the extensibility layer of GuardianIQ. It allows the platform to express new governance graph edges without redesigning the database every time a new object type is added.

Column	Type	Purpose	Required
id	UUID	Primary key	Yes
tenant_id	UUID	Tenant boundary	Yes
source_type	VARCHAR	Source object type	Yes
source_id	UUID / VARCHAR	Source identifier	Yes
relationship_type	VARCHAR	Semantic relationship	Yes
target_type	VARCHAR	Target object type	Yes
target_id	UUID / VARCHAR	Target identifier	Yes
relationship_scope	VARCHAR / JSONB	Where relationship applies	Recommended
responsibility_type	VARCHAR	OWNER / REVIEWER / APPROVER etc.	Conditional
effective_from	TIMESTAMPTZ	Start validity	Yes
effective_to	TIMESTAMPTZ	End validity	Optional
status	VARCHAR	PROPOSED / ACTIVE etc.	Yes
metadata_json	JSONB	Additional relationship context	Yes
approved_by	UUID	Approval lineage	Conditional

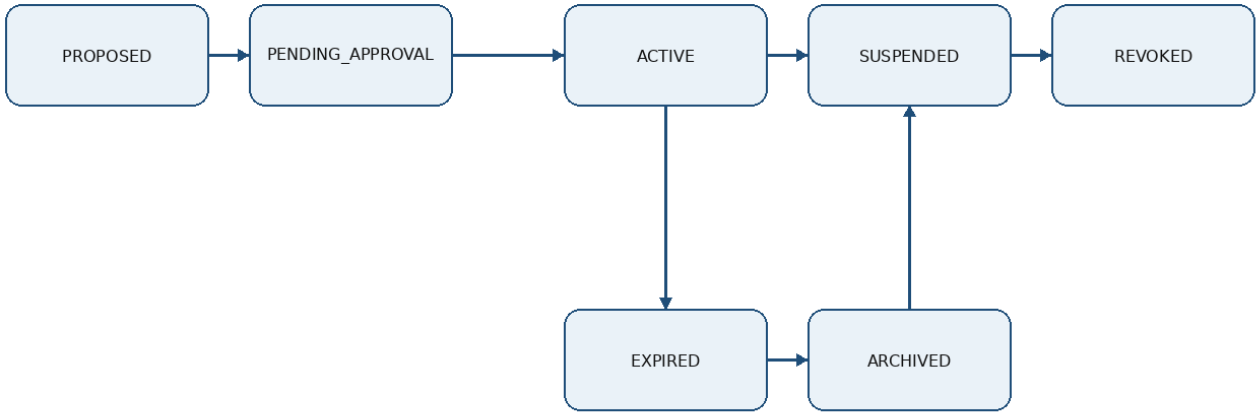
Section G - Specific vs Generic Relationship Strategy

Use Specific Mapping Table When	Use Generic Relationship Table When	Example
High-volume joins and strong constraints are required.	Relationship is extensible, exploratory or lower volume.	Specific: ai_agent_tool_mapping; Generic: evidence supports decision.
The relationship drives core runtime authorization.	The relationship supports graph exploration or traceability.	Specific: user_role_mapping; Generic: policy influenced recommendation.
FK integrity must be enforced strongly.	Target object type may vary significantly.	Specific: workflow_schedule_agent_assignment; Generic: object relationship graph.
The UI frequently filters or aggregates by the relationship.	The relationship is primarily used for lineage or audit context.	Specific: user_group_membership; Generic: derived_from.

Engineering rule: Do not use generic_relationships as a dumping ground. Critical operational relationships should use specific tables with constraints. Generic relationships should complement them for flexible governance graph traversal.

Section H - Relationship Lifecycle and State Management

Relationship Lifecycle State Machine



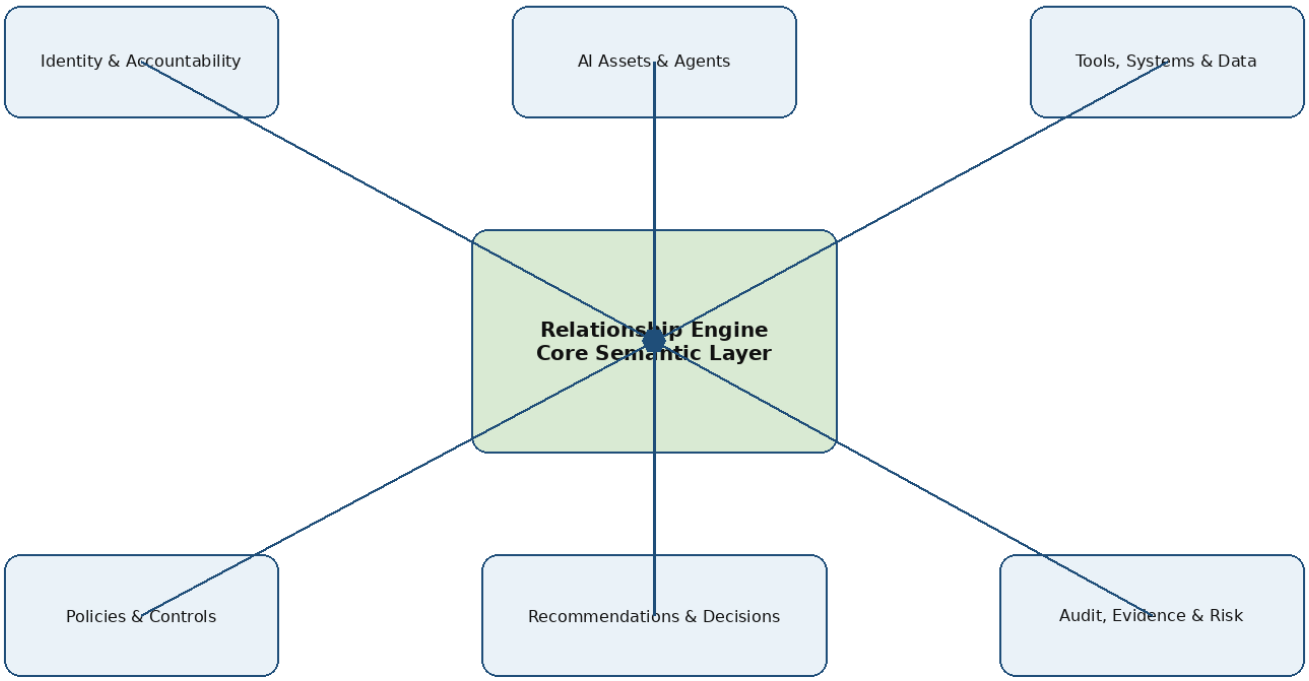
Rule: production relationships are expired, suspended or revoked - not hard deleted.

State	Meaning	Allowed Next States	Example
PROPOSED	Relationship requested but not active.	PENDING_APPROVAL, ACTIVE, REVOKED	Agent requests access to ERP pricing API.
PENDING_APPROVAL	Waiting for governance approval.	ACTIVE, REJECTED, REVOKED	High-risk tool access awaiting Risk Manager.
ACTIVE	Relationship currently valid.	SUSPENDED, REVOKED, EXPIRED	Agent can read registry data.
SUSPENDED	Temporarily disabled.	ACTIVE, REVOKED	Access suspended due to incident.
REVOKED	Explicitly removed.	ARCHIVED	Tool access revoked.
EXPIRED	Ended due to date.	ARCHIVED	Temporary delegation ended.
ARCHIVED	Retained historical record.	None	Retired workflow-policy binding retained.

Section I - Relationship Engine Architecture

The Relationship Engine is a shared backend capability. It should not be embedded separately inside each module. All modules should call the relationship service for creating, resolving, validating, revoking and traversing relationship records.

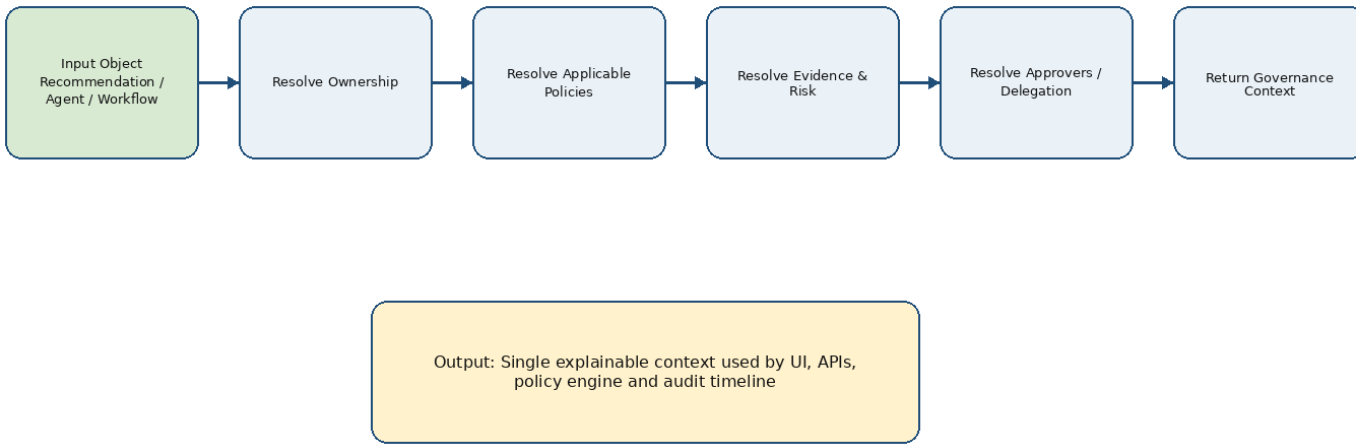
GuardianIQ Relationship Architecture - Volume 1



Component	Responsibility
Relationship API Layer	Receives create, update, revoke, search and graph traversal requests.
Relationship Service	Orchestrates validation, persistence, event publishing and response shaping.
Relationship Repository	Manages specific and generic relationship queries.
Relationship Resolver	Returns active relationships for an object within scope and time window.
Graph Builder	Builds relationship graph for explainability and UI explorer.
Validation Engine	Prevents invalid, duplicate, circular, orphan or unauthorized relationships.
Event Publisher	Creates immutable audit events for relationship lifecycle changes.
Cache Layer	Caches stable read-heavy relationships such as role permissions and registry ownership.
Authorization Service	Verifies RBAC, ABAC and relationship checks before sensitive operations.

Section J - Relationship Resolution and Graph Traversal

Relationship Resolution Engine



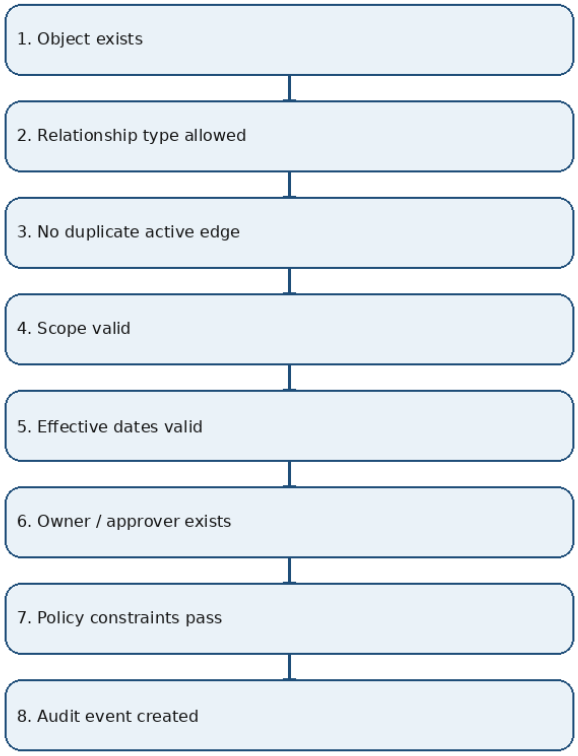
Relationship resolution pseudocode

```
def resolve_governance_context(object_type, object_id, scope=None, as_of=None):
    object_ref = object_repo.get(object_type, object_id)
    owners = relationship_repo.find_active(object_type, object_id, 'OWNED_BY', as_of)
    policies = relationship_repo.find_targets(object_type, object_id, 'GOVERNED_BY', scope, as_of)
    evidence = relationship_repo.find_targets(object_type, object_id, 'SUPPORTED_BY', scope, as_of)
    risks = risk_repo.find_for_object(object_type, object_id)
    approvals = approval_repo.find_for_object(object_type, object_id)
    events = audit_repo.find_timeline(object_type, object_id)
    return GovernanceContext(object_ref, owners, policies, evidence, risks, approvals, events)
```

Traversal Type	Purpose	Depth	Example
Immediate neighbors	Show direct relationships around an object.	1 hop	AI Agent -> Tools, Model, Owner, Workflows
Governance context	Resolve ownership, policies, approvals, evidence and audit.	2-3 hops	Recommendation -> Policy Evaluation -> Approval -> Decision
Impact analysis	Find downstream objects affected by a change.	Variable	Retire model -> affected agents, workflows, schedules
Audit reconstruction	Recreate lifecycle from events and relationships.	Timeline-based	Who changed agent tool access and when?
Policy applicability	Find policies binding to object or inherited context.	1-3 hops	Workflow inherits BU risk policy

Section K - Validation Engine and Rule Catalogue

Relationship Validation Engine



Rule ID	Category	Validation Rule	Failure Behavior
REL-VAL-001	Ownership	Object must have at least one active OWNER before activation.	Block save/activation
REL-VAL-002	Ownership	High-risk object must have OWNER and APPROVER.	Block save/activation
REL-VAL-003	Ownership	Owner must be active user/group/department.	Block save/activation
REL-VAL-004	Ownership	Owner cannot be deleted while active object exists.	Block save/activation
REL-VAL-005	Ownership	Multiple owners require primary owner marker.	Block save/activation
REL-VAL-006	Duplicates	No duplicate active relationship for same source/type/target/scope.	Block save/activation
REL-VAL-007	Duplicates	Effective dates cannot overlap for exclusive relationship types.	Block save/activation
REL-VAL-008	Duplicates	Same approver cannot appear twice in same approval group assignment.	Block save/activation
REL-VAL-009	Temporal	effective_to must be greater than effective_from.	Block save/activation
REL-VAL-010	Temporal	Expired relationship cannot authorize action.	Block save/activation
REL-VAL-011	Temporal	Future-dated relationship cannot be used until effective_from.	Block save/activation
REL-VAL-012	Temporal	Delegation must fall inside allowed window.	Block save/activation
REL-VAL-013	Authorization	User must have permission to create relationship.	Block save/activation
REL-VAL-014	Authorization	Activation requires approval if relationship increases risk.	Block save/activation
REL-VAL-015	Authorization	Tool execute access requires explicit permission.	Block save/activation
REL-VAL-016	Authorization	External auditor relationship must have expiry.	Block save/activation
REL-VAL-017	AI Agent Boundaries	Agent cannot use tool without active USES_TOOL.	Block save/activation
REL-VAL-018	AI Agent Boundaries	Requested operation must match access_mode.	Block save/activation
REL-VAL-019	AI Agent Boundaries	Agent execution mode cannot exceed registry mode.	Block save/activation
REL-VAL-020	AI Agent Boundaries	Blocked operation overrides allowed operation.	Block save/activation
REL-VAL-021	Policy	Workflow activation requires at least one active policy binding where configured.	Block save/activation
REL-VAL-022	Policy	Policy binding target must be valid governable object.	Block save/activation
REL-VAL-023	Policy	Policy version must be active or approved.	Block save/activation
REL-VAL-024	Evidence	High-risk recommendation requires evidence link.	Block save/activation
REL-VAL-025	Evidence	Evidence source must be known or explicitly marked external.	Block save/activation
REL-VAL-026	Evidence	Evidence sensitivity must not exceed viewer access scope.	Block save/activation
REL-VAL-027	Lifecycle	REVOKED relationship cannot return to ACTIVE.	Block save/activation
REL-VAL-028	Lifecycle	ARCHIVED relationship cannot be updated except metadata for retention.	Block save/activation
REL-VAL-029	Lifecycle	Suspended relationship cannot authorize runtime execution.	Block save/activation
REL-VAL-030	Graph Integrity	Circular ownership must be blocked.	Block save/activation
REL-VAL-031	Graph Integrity	Orphan relationship target must be blocked.	Block save/activation
REL-VAL-032	Graph Integrity	Cross-tenant relationship must be blocked unless explicitly allowed.	Block save/activation
REL-VAL-033	Audit	Every relationship state change must create governance event.	Log failure and block transaction
REL-VAL-034	Audit	Reason is mandatory for revoke, override, emergency delegation and suspension.	Log failure and block transaction

Section L - Security, RBAC, ABAC and Accountability

Relationship operations are sensitive because they can change who owns, approves, accesses, executes or evidences a governed object. GuardianIQ must apply authorization to relationship changes, not just object CRUD.

Action	RBAC Permission	ABAC / Relationship Checks
Create ownership relationship	CREATE_RELATIONSHIP	User is governance admin or object owner within department scope.
Assign AI agent to tool	ASSIGN_AGENT_TOOL	Agent owner or governance admin; tool sensitivity within user scope.
Activate relationship	ACTIVATE_RELATIONSHIP	Approver exists if high-risk; relationship does not violate policy.
Revoke relationship	REVOKE_RELATIONSHIP	Owner, governance admin or escalation owner with reason.
View relationship graph	VIEW_GOVERNANCE_GRAPH	Scope-limited by object sensitivity, BU, department and role.
Export relationship evidence	EXPORT_AUDIT_EVIDENCE	Auditor/compliance role; export event must be logged.

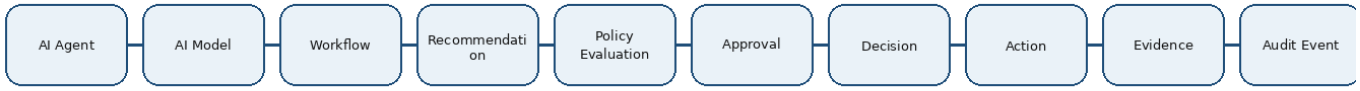
Authorization decision pattern

```
Authorization Decision = RBAC Permission + ABAC Context + Relationship Check

Subject attributes: user_id, roles, department, business_unit, approval_limit, external_access_scope
Object attributes: object_type, owner_department, risk_level, data_sensitivity, lifecycle_state
Action attributes: CREATE_RELATIONSHIP, REVOKE_RELATIONSHIP, VIEW_GRAPH, ACTIVATE_RELATIONSHIP
Environment attributes: time, emergency_flag, delegation_active, tenant_id, request_source
Relationship attributes: owner, reviewer, approval_group_member, delegated_from, effective_dates
```

Section M - Audit, Provenance and Evidence Model

Traceability and Provenance Chain



Goal: reconstruct who/what generated, reviewed, approved, acted, evidenced and audited every governed AI decisio

Event Code	When Generated	Required Payload
RELATIONSHIP_CREATED	New relationship record is created.	source, target, type, scope, actor, reason
RELATIONSHIP_UPDATED	Relationship metadata or scope changes.	before_json, after_json, changed_fields

RELATIONSHIP_APPROVED	Pending relationship is approved.	approver, approval_group, decision_reason
RELATIONSHIP_ACTIVATED	Relationship becomes active.	effective_from, status, approval_reference
RELATIONSHIP_SUSPENDED	Relationship temporarily disabled.	reason, suspended_by, expected_resolution
RELATIONSHIP_REVOKED	Relationship permanently revoked.	reason, revoked_by, impact_summary
RELATIONSHIP_EXPIRED	Relationship reaches effective_to.	expiration_time, system_actor
RELATIONSHIP_VALIDATION_FAILED	Invalid relationship attempted.	rule_id, failure_reason, attempted_payload
GRAPH_TRAVERSAL_PERFORMED	Sensitive graph query executed.	requester, object_ref, depth, purpose

Section N - Logical Data Model Guidance

Logical Entity	Purpose	Recommended Physical Pattern	Phase
object_responsibilities	Owner/reviewer/approver assignments.	Specific table + generic graph mirror	3
generic_relationships	Flexible governance graph edges.	Generic table with indexed source/target/type/scope	3
relationship_events	Relationship lifecycle audit.	audit.audit_events with entity references	3
agent_tool_permissions	Runtime authorization.	Specific mapping table	1-3
workflow_policy_bindings	Policy applicability.	Specific table or generic GOVERNED_BY relationship	3-4
evidence_links	Evidence to object.	Generic evidence_link table	4
policy_evaluations	Policy result history.	Specific evaluation table	4
approval_assignments	Approver routing.	Specific table	4
graph_snapshots	Optional read model for impact analysis.	Materialized view / cache table	Future

Section O - API Architecture and Contract Patterns

Method	Endpoint	Purpose	Permission
GET	/api/v1/relationships	List/search relationships	VIEW_RELATIONSHIP
POST	/api/v1/relationships	Create relationship	CREATE_RELATIONSHIP
GET	/api/v1/relationships/{id}	Get relationship detail	VIEW_RELATIONSHIP
PUT	/api/v1/relationships/{id}	Update relationship metadata/scope	UPDATE_RELATIONSHIP
POST	/api/v1/relationships/{id}/submit	Submit relationship for approval	SUBMIT_RELATIONSHIP
POST	/api/v1/relationships/{id}/approve	Approve pending relationship	APPROVE_RELATIONSHIP
POST	/api/v1/relationships/{id}/activate	Activate relationship	ACTIVATE_RELATIONSHIP
POST	/api/v1/relationships/{id}/suspend	Suspend relationship	SUSPEND_RELATIONSHIP
POST	/api/v1/relationships/{id}/revoke	Revoke relationship	REVOKE_RELATIONSHIP
GET	/api/v1/objects/{type}/{id}/relationships	Get all relationships for object	VIEW_RELATIONSHIP
GET	/api/v1/objects/{type}/{id}/governance-context	Resolved governance context	VIEW_GOVERNANCE_CONTEXT
GET	/api/v1/graph/{type}/{id}	Graph traversal	VIEW_GOVERNANCE_GRAPH
POST	/api/v1/relationships/validate	Validate prospective relationship	VALIDATE_RELATIONSHIP
GET	/api/v1/responsibilities	Find responsibility assignments	VIEW_RESPONSIBILITY
POST	/api/v1/responsibilities	Create responsibility assignment	CREATE_RESPONSIBILITY
GET	/api/v1/relationships/types	List relationship types	VIEW_REFERENCE_DATA
GET	/api/v1/relationships/lifecycle-states	List lifecycle states	VIEW_REFERENCE_DATA

Relationship API response example


```
{
  "success": true,
  "data": {
    "object": {
      "type": "AI_AGENT",
      "id": "AGT-GOV-001"
    },
    "relationships": [
      {
        "type": "OWNED_BY",
        "target_type": "USER",
        "target_id": "USR-001",
        "status": "ACTIVE"
      },
      {
        "type": "USES_MODEL",
        "target_type": "AI_MODEL",
        "target_id": "MOD-LLM-001",
        "status": "ACTIVE"
      },
```

```
      {
        "type": "USES_TOOL",
        "target_type": "TOOL",
        "target_id": "TOOL-REGISTRY-READ",
        "metadata": {
          "access_mode": "READ_ONLY"
        }
      }
    ]
  },
  "error": null,
  "request_id": "REQ-20260615-001"
}
```

Section P - Backend Service Implementation Guide

Module	Files	Responsibility
relationship	models.py, schemas.py, repository.py, service.py, validators.py	Core relationship CRUD, validation and lifecycle.
responsibility	models.py, service.py	Owner/reviewer/approver assignments.
graph	graph_builder.py, traversal.py	Governance graph traversal and context building.
validation	rule_catalog.py, engine.py	Reusable validation engine for relationship rules.
authorization	decision_service.py	RBAC + ABAC + relationship checks.
audit	event_service.py	Event publishing and timeline integration.
cache	cache_keys.py, cache_service.py	Cache stable relationship lookups.
integration	adapters.py	Allow future sync with external systems or graph DB.

Recommended backend folder structure

```
backend/app/modules/relationship/
  models.py              # SQLAlchemy models
  schemas.py             # Pydantic request/response models
  repository.py          # DB persistence and query patterns
  service.py             # Business orchestration
  validators.py          # Domain validation
  lifecycle.py           # State transitions
  graph_builder.py       # Object graph output
  resolver.py            # Governance context resolver
  api.py                 # FastAPI routes
```

Section Q - Frontend and UI/UX Reference

Volume 2 should contain the full screen-by-screen UI/UX specification. This section defines enough relationship-aware UX requirements for backend and frontend developers to remain aligned.

Screen	Primary Purpose	Relationship Data Needed
Relationship Explorer	Search and browse relationships across object types.	source, target, type, lifecycle, scope, owner.
Governance Graph View	Visualize connected objects around a selected object.	nodes, edges, labels, risk levels, edge status.
Create Relationship Wizard	Create specific or generic relationships.	available object types, allowed relationship types, validation result.
Object Relationship Panel	Embedded view on AI model/agent/workflow detail pages.	direct relationships + important derived relationships.
Responsibility Assignment Panel	Assign owners, reviewers, approvers and auditors.	actor list, responsibility types, effective dates.
Relationship Timeline	Show relationship history and events.	events, before/after changes, actor, reason.
Impact Analysis View	Show what changes if object or relationship changes.	downstream graph, affected workflows, policies and agents.

Section R - Performance, Caching and Scaling

Concern	Recommendation	Reason
Source/target lookup	Composite indexes on tenant_id, source_type, source_id, relationship_type, status	Fast object relationship retrieval.
Target reverse lookup	Composite index on tenant_id, target_type, target_id, relationship_type	Impact analysis and graph traversal.
JSONB metadata	Use GIN indexes only for fields queried frequently	Avoid write overhead from unnecessary indexes.
High-volume audit	Partition audit events by time or tenant when volume grows	Maintain audit performance.
Graph traversal	Limit depth and node count in API	Prevent expensive uncontrolled graph queries.
Caching	Cache stable relationships such as role permissions, ownership and policy bindings	Reduce repeated joins.
Materialized views	Use for dashboard or graph summaries, not transaction truth	Keep operational truth in normalized tables.
Tenant isolation	Include tenant_id in every index and query	Security and performance.

Section S - Enterprise Examples and Scenarios

ID	Scenario	Domain	Relationship Lesson
EX-01	AI agent tool access	AI Governance	Agent uses registry read API but is blocked from write tool execution.
EX-02	Model retirement impact	AI Asset Lifecycle	Retiring a model identifies affected agents, workflows and schedules.
EX-03	Delegated approval	Identity	Approval relationship temporarily routes from original approver to delegate.
EX-04	Pharma RFQ governance	Pharma Commercial	Recommendation is linked to workflow, evidence, policy, approver, decision and action.
EX-05	BFSI high-risk recommendation	BFSI	High-risk recommendation requires risk manager and compliance approval.
EX-06	Manufacturing support intelligence	Manufacturing	Issue cluster generated by agent links to product family and resolution workflow.
EX-07	Policy violation remediation	Compliance	Agent attempts blocked operation, violation creates remediation action.
EX-08	Evidence lineage	Audit	Decision is supported by documents, signals and model explanation.
EX-09	External auditor scope	Audit	External auditor can view only scoped BU and time-bound evidence.
EX-10	Cross-department ownership	Operations	Workflow owned by Sales but reviewed by Compliance.
EX-11	Tool sensitivity restriction	Security	Restricted tool requires approval relationship before use.
EX-12	Workflow policy inheritance	Policy	Workflow inherits policy from business unit and data sensitivity.
EX-13	Emergency override	Operations	Override creates relationship event, reason and post-facto review.
EX-14	Duplicate relationship prevention	Data Quality	Duplicate active ownership edge is blocked.
EX-15	Broken graph detection	Integrity	Orphan relationship target is rejected.
EX-16	AI model version lineage	AI Lifecycle	Recommendation records model version and deployment context.
EX-17	Scheduled governance review	Workflow	Daily schedule generates findings and audit events.
EX-18	High-risk evidence requirement	Risk	Recommendation cannot be approved without supporting evidence.
EX-19	Approval group membership change	Identity	New group member can approve only after effective_from.
EX-20	Data source classification change	Data Governance	Classification change triggers impact analysis for workflows and agents.
EX-21	Tool access suspension	Security	Tool access suspended after violation; future runs blocked.
EX-22	Policy version change	Policy	New version affects only future evaluations unless backtesting requested.
EX-23	Action execution ownership	Execution	Approved decision creates action assigned to human owner.
EX-24	Machine actor accountability	AI Governance	AI agent is treated as actor with owner and execution mode.
EX-25	Governance graph explorer	Product UX	User opens object and sees connected owners, policies, evidence and events.
EX-26	SignalIQ integration	Integration	SignalIQ recommendation becomes GuardianIQ governable object.
EX-27	GuardianIQ self-governance	Platform	Changes to GuardianIQ agent permissions are themselves governed.
EX-28	Customer-specific policy binding	Multi-tenant	Tenant-specific policy governs same workflow differently.
EX-29	Relationship expiry	Lifecycle	Temporary relationship expires and no longer authorizes execution.
EX-30	Audit reconstruction	Audit	Auditor reconstructs full lifecycle from events and relationships.

Detailed scenario example

```
Scenario: Agent tool access violation

Given:
  AI_AGENT AGT-GOV-001 USES_TOOL TOOL-ERP-QUOTE
  metadata.access_mode = READ_ONLY
When:
  Agent requests operation EXECUTE_QUOTE_CREATION
Then:
  Relationship validation blocks execution
  Event AI_ACTION_BLOCKED is generated
  Policy violation is created if the attempted operation is sensitive
  Escalation notification is routed to agent owner and governance admin
```

Section T - Implementation Roadmap, QA and Definition of Done

Step	Activity	Primary Owner	Output
1	Approve relationship taxonomy and lifecycle states.	Product Architect	Approved taxonomy.
2	Create generic relationship schema and key specific mapping schemas.	Backend/Data Engineer	Migration scripts.
3	Implement relationship repository and service layer.	Backend Developer	Reusable backend module.
4	Implement validation engine.	Backend Developer + QA	Validation rule catalogue in code.
5	Implement relationship APIs.	Backend Developer	OpenAPI documented endpoints.
6	Implement audit event publishing.	Backend Developer	Events for every state change.
7	Implement relationship explorer UI foundation.	Frontend Developer	Search/browse relationships.
8	Implement object relationship panels.	Frontend Developer	Embedded relationship context.
9	Implement graph view MVP.	Frontend Developer	Node-edge graph for selected object.
10	Execute QA matrix and UAT scenarios.	QA + PM	Signed test evidence and DoD.

Test ID	Scenario	Expected Result	Evidence
TC-001	Create active ownership relationship	Pass / Fail	Evidence screenshot or API response
TC-002	Block relationship without valid source object	Pass / Fail	Evidence screenshot or API response
TC-003	Block cross-tenant relationship	Pass / Fail	Evidence screenshot or API response
TC-004	Block duplicate active relationship	Pass / Fail	Evidence screenshot or API response
TC-005	Expire relationship and verify no longer authorizes action	Pass / Fail	Evidence screenshot or API response
TC-006	Suspend agent tool access and verify run blocked	Pass / Fail	Evidence screenshot or API response
TC-007	Resolve governance context for AI Agent	Pass / Fail	Evidence screenshot or API response
TC-008	Graph traversal depth limit enforced	Pass / Fail	Evidence screenshot or API response
TC-009	Relationship revocation requires reason	Pass / Fail	Evidence screenshot or API response
TC-010	Relationship lifecycle event created	Pass / Fail	Evidence screenshot or API response
TC-011	Approval required for high-risk tool access	Pass / Fail	Evidence screenshot or API response
TC-012	Delegation relationship works within date range	Pass / Fail	Evidence screenshot or API response
TC-013	Delegation relationship fails outside date range	Pass / Fail	Evidence screenshot or API response
TC-014	Evidence link visible to auditor	Pass / Fail	Evidence screenshot or API response
TC-015	Evidence link hidden from unauthorized user	Pass / Fail	Evidence screenshot or API response
TC-016	Impact analysis identifies downstream workflow	Pass / Fail	Evidence screenshot or API response
TC-017	Policy binding discovered for workflow	Pass / Fail	Evidence screenshot or API response
TC-018	Circular ownership blocked	Pass / Fail	Evidence screenshot or API response
TC-019	Orphan relationship target blocked	Pass / Fail	Evidence screenshot or API response
TC-020	API pagination works	Pass / Fail	Evidence screenshot or API response
TC-021	API filtering by source works	Pass / Fail	Evidence screenshot or API response
TC-022	API reverse lookup by target works	Pass / Fail	Evidence screenshot or API response
TC-023	Bulk graph export restricted	Pass / Fail	Evidence screenshot or API response
TC-024	Audit reconstruction shows actor and reason	Pass / Fail	Evidence screenshot or API response
TC-025	Relationship metadata update creates change log	Pass / Fail	Evidence screenshot or API response
TC-026	Performance under 10k relationships acceptable	Pass / Fail	Evidence screenshot or API response
TC-027	Cache invalidation after update works	Pass / Fail	Evidence screenshot or API response
TC-028	No hard delete in production path	Pass / Fail	Evidence screenshot or API response
TC-029	Soft delete excludes from active queries	Pass / Fail	Evidence screenshot or API response
TC-030	Relationship explorer UI shows empty state	Pass / Fail	Evidence screenshot or API response

Definition of Done: Volume 1 implementation is complete only when the relationship service, validation engine, APIs, audit events, core UI views, QA test cases and developer documentation are all working against integrated Phase 0, Phase 1 and Phase 2 data.

Section U - Appendices

Appendix A - Naming Standards

Item	Pattern	Example
Relationship type	UPPER_SNAKE_CASE verb phrase	USES_TOOL
Object type	UPPER_SNAKE_CASE noun	AI_AGENT
Relationship code	REL-YYYY-NNNNN	REL-2026-00042
Event code	UPPER_SNAKE_CASE past-tense/action	RELATIONSHIP_REVOKED
API resource	plural kebab-case	/relationships
Database table	snake_case plural	generic_relationships

Appendix B - Glossary

Term	Meaning
Governable Object	Any object under AI governance control.
Relationship	A semantic connection between two actors or governable objects.
Specific Mapping	A dedicated table for a high-value relationship.

Generic Relationship	Flexible table-driven edge in the governance graph.
Relationship Scope	Boundary where a relationship is valid.
Relationship Resolver	Service that returns relevant relationships for an object.
Graph Traversal	Process of walking connected relationships to understand context or impact.
Provenance	Traceable origin and influence chain behind an artifact or decision.
Responsibility Type	OWNER, REVIEWER, APPROVER, EXECUTOR, AUDITOR, ESCALATION_OWNER.
Policy Binding	Relationship connecting policy to object/context it governs.

Appendix C - Reference Sources

Source	Use in Handbook
GuardianIQ Step 3 - Relationship Architecture Foundation	Primary source for relationship taxonomy, principles and phase scope.
NIST AI Risk Management Framework	Directional alignment for governance and AI lifecycle risk management.
ISO/IEC 42001:2023	Directional alignment for AI management system concepts.
EU AI Act official overview	Directional alignment for risk-based oversight, logging and human supervision.
W3C PROV-O	Directional alignment for provenance, agents, entities and activities.

Final Recommendation: Treat this Volume 1 as the canonical relationship architecture source for GuardianIQ. Do not redefine relationship meanings in UI, backend, API or database volumes. Later volumes should specialize implementation details while preserving the semantics defined here.

Part 2 - Detailed Enterprise Relationship Implementation Reference

This part expands Volume 1 into a practical engineering reference. It is intended to remove open dependencies for developers by making relationship semantics, data behaviors, service responsibilities, validation, API contracts and QA expectations explicit.

Detailed Logical Table Specifications

generic_relationships

Column	Type	Description
id	UUID	Primary key
tenant_id	UUID	Tenant boundary
source_type	VARCHAR(100)	Source object type
source_id	UUID/VARCHAR	Source object id
relationship_type	VARCHAR(100)	Semantic relationship
target_type	VARCHAR(100)	Target object type
target_id	UUID/VARCHAR	Target object id
relationship_scope	VARCHAR(255)	Scope key such as WORKFLOW:WF-001
scope_json	JSONB	Structured scope
responsibility_type	VARCHAR(100)	Optional OWNER/REVIEWER/APPROVER
effective_from	TIMESTAMPTZ	Validity start
effective_to	TIMESTAMPTZ	Validity end
status	VARCHAR(50)	Lifecycle state
metadata_json	JSONB	Relationship attributes
created_by	UUID	Creator
approved_by	UUID	Approver
created_at	TIMESTAMPTZ	Creation timestamp
updated_at	TIMESTAMPTZ	Update timestamp

Implementation note: generic_relationships should include tenant_id in all queries, created/updated timestamps, soft-delete where relevant, and audit event generation for critical changes.

object_responsibilities

Column	Type	Description
id	UUID	Primary key
tenant_id	UUID	Tenant boundary
object_type	VARCHAR(100)	Governable object type
object_id	UUID/VARCHAR	Governable object id
actor_type	VARCHAR(50)	USER/GROUP/AGENT/DEPARTMENT
actor_id	UUID/VARCHAR	Actor id
responsibility_type	VARCHAR(50)	OWNER/REVIEWER/APPROVER/EXECUTOR/AUDITOR
is_primary	BOOLEAN	Primary owner flag
effective_from	TIMESTAMPTZ	Validity start
effective_to	TIMESTAMPTZ	Validity end
status	VARCHAR(50)	ACTIVE/REVOKED/EXPIRED
metadata_json	JSONB	Context

Implementation note: object_responsibilities should include tenant_id in all queries, created/updated timestamps, soft-delete where relevant, and audit event generation for critical changes.

relationship_validation_results

Column	Type	Description
id	UUID	Primary key
tenant_id	UUID	Tenant boundary
request_id	VARCHAR(150)	Correlation id
relationship_id	UUID	Optional relationship
validation_rule_id	VARCHAR(50)	Rule id
validation_status	VARCHAR(50)	PASSED/FAILED/WARNING
severity	VARCHAR(50)	LOW/MEDIUM/HIGH/CRITICAL
message	TEXT	Developer-readable message
resolution_hint	TEXT	Suggested fix
payload_json	JSONB	Input evaluated
created_at	TIMESTAMPTZ	Timestamp

Implementation note: relationship_validation_results should include tenant_id in all queries, created/updated timestamps, soft-

delete where relevant, and audit event generation for critical changes.

relationship_graph_snapshots

Column	Type	Description
id	UUID	Primary key
tenant_id	UUID	Tenant boundary
root_object_type	VARCHAR(100)	Root type
root_object_id	UUID/VARCHAR	Root id
depth	INTEGER	Traversal depth
snapshot_json	JSONB	Nodes and edges
generated_by	UUID	User/system
created_at	TIMESTAMPTZ	Timestamp
expires_at	TIMESTAMPTZ	Cache expiry

Implementation note: relationship_graph_snapshots should include tenant_id in all queries, created/updated timestamps, soft-delete where relevant, and audit event generation for critical changes.

policy_bindings

Column	Type	Description
id	UUID	Primary key
tenant_id	UUID	Tenant boundary
policy_id	UUID	Policy
target_type	VARCHAR(100)	Target object type
target_id	UUID/VARCHAR	Target object id
binding_scope	VARCHAR(255)	Where binding applies
priority	INTEGER	Policy priority
is_mandatory	BOOLEAN	Mandatory flag
effective_from	TIMESTAMPTZ	Start
effective_to	TIMESTAMPTZ	End
status	VARCHAR(50)	Lifecycle

Implementation note: policy_bindings should include tenant_id in all queries, created/updated timestamps, soft-delete where relevant, and audit event generation for critical changes.

evidence_links

Column	Type	Description
id	UUID	Primary key
tenant_id	UUID	Tenant boundary
evidence_id	UUID/VARCHAR	Evidence artifact
target_type	VARCHAR(100)	Target object
target_id	UUID/VARCHAR	Target object id
link_type	VARCHAR(100)	SUPPORTS/CONTRADICTS/EXPLAINS
confidence_score	NUMERIC	Optional confidence
source_system	VARCHAR(150)	Source system
metadata_json	JSONB	Evidence context

Implementation note: evidence_links should include tenant_id in all queries, created/updated timestamps, soft-delete where relevant, and audit event generation for critical changes.

Detailed API Contract Catalogue

Relationship CRUD APIs

Method	Endpoint	Purpose
POST	/api/v1/relationships	Create a generic relationship
GET	/api/v1/relationships	List relationships
GET	/api/v1/relationships/{relationship_id}	Get detail
PUT	/api/v1/relationships/{relationship_id}	Update metadata/scope
POST	/api/v1/relationships/{relationship_id}/revoke	Revoke relationship
POST	/api/v1/relationships/{relationship_id}/suspend	Suspend relationship
POST	/api/v1/relationships/{relationship_id}/expire	Expire relationship

Standard response envelope

```
{
  "success": true,
  "data": {
    "message": "API response must use common envelope",
    "items": [],
    "page": 1,
    "page_size": 25
  },
  "error": null,
  "request_id": "REQ-20260615-0001",
  "timestamp": "2026-06-15T09:00:00Z"
}
```

Responsibility APIs

Method	Endpoint	Purpose
POST	/api/v1/responsibilities	Assign responsibility
GET	/api/v1/responsibilities	Search responsibility assignments
POST	/api/v1/responsibilities/{id}/revoke	Revoke responsibility
GET	/api/v1/objects/{type}/{id}/owners	Resolve active owners
GET	/api/v1/objects/{type}/{id}/approvers	Resolve active approvers

Standard response envelope

```
{
  "success": true,
  "data": {
    "message": "API response must use common envelope",
    "items": [],
    "page": 1,
    "page_size": 25
  },
  "error": null,
  "request_id": "REQ-20260615-0001",
  "timestamp": "2026-06-15T09:00:00Z"
}
```

Graph APIs

Method	Endpoint	Purpose
GET	/api/v1/graph/{object_type}/{object_id}	Return relationship graph
POST	/api/v1/graph/impact-analysis	Analyze downstream impact
GET	/api/v1/graph/{object_type}/{object_id}/timeline	Timeline graph
POST	/api/v1/graph/export	Export graph for audit

Standard response envelope

```
{
  "success": true,
  "data": {
    "message": "API response must use common envelope",
    "items": [],
    "page": 1,
    "page_size": 25
```

```
  },
  "error": null,
  "request_id": "REQ-20260615-0001",
  "timestamp": "2026-06-15T09:00:00Z"
}
```

Validation APIs

Method	Endpoint	Purpose
POST	/api/v1/relationships/validate	Validate one relationship payload
POST	/api/v1/relationships/bulk-validate	Validate many relationship payloads
GET	/api/v1/validation-rules	List active validation rules

Standard response envelope

```
{
  "success": true,
  "data": {
    "message": "API response must use common envelope",
    "items": [],
    "page": 1,
    "page_size": 25
  },
  "error": null,
  "request_id": "REQ-20260615-0001",
  "timestamp": "2026-06-15T09:00:00Z"
}
```

Resolver APIs

Method	Endpoint	Purpose
GET	/api/v1/objects/{type}/{id}/governance-context	Resolve full governance context
GET	/api/v1/objects/{type}/{id}/applicable-policies	Resolve policies
GET	/api/v1/objects/{type}/{id}/evidence	Resolve evidence
GET	/api/v1/objects/{type}/{id}/audit-trail	Resolve audit timeline

Standard response envelope

```
{
  "success": true,
  "data": {
    "message": "API response must use common envelope",
    "items": [],
    "page": 1,
    "page_size": 25
  },
  "error": null,
  "request_id": "REQ-20260615-0001",
  "timestamp": "2026-06-15T09:00:00Z"
}
```

Backend Service Detailed Design

RelationshipService

Public Method	Purpose
create_relationship	Implements create relationship capability.
update_relationship	Implements update relationship capability.
submit_for_approval	Implements submit for approval capability.
activate_relationship	Implements activate relationship capability.
suspend_relationship	Implements suspend relationship capability.
revoke_relationship	Implements revoke relationship capability.
expire_relationship	Implements expire relationship capability.
search_relationships	Implements search relationships capability.

Dependencies: ValidationEngine, RelationshipRepository, AuthorizationService, AuditEventService

RelationshipResolverService

Public Method	Purpose
resolve_direct	Implements resolve direct capability.
resolve_reverse	Implements resolve reverse capability.
resolve_governance_context	Implements resolve governance context capability.
resolve_owners	Implements resolve owners capability.
resolve_policies	Implements resolve policies capability.
resolve_evidence	Implements resolve evidence capability.

Dependencies: RelationshipRepository, ObjectReferenceService, CacheService

GraphService

Public Method	Purpose
build_graph	Implements build graph capability.
impact_analysis	Implements impact analysis capability.
timeline_graph	Implements timeline graph capability.
export_graph	Implements export graph capability.

Dependencies: RelationshipResolverService, AuditService, PermissionService

ResponsibilityService

Public Method	Purpose
assign_owner	Implements assign owner capability.
assign_reviewer	Implements assign reviewer capability.
assign_approver	Implements assign approver capability.
resolve_responsibilities	Implements resolve responsibilities capability.
revoke_assignment	Implements revoke assignment capability.

Dependencies: RelationshipService, IdentityService

ValidationEngine

Public Method	Purpose
validate_payload	Implements validate payload capability.
validate_lifecycle_transition	Implements validate lifecycle transition capability.
validate_relationship_uniqueness	Implements validate relationship uniqueness capability.
validate_scope	Implements validate scope capability.
validate_authorization	Implements validate authorization capability.

Dependencies: RuleCatalog, RelationshipRepository, AuthorizationService

RelationshipAuditService

Public Method	Purpose
publish_created	Implements publish created capability.
publish_updated	Implements publish updated capability.

publish_revoked	Implements publish revoked capability.
publish_validation_failure	Implements publish validation failure capability.
get_timeline	Implements get timeline capability.

Dependencies: AuditEventRepository

RelationshipCacheService

Public Method	Purpose
get_cached_context	Implements get cached context capability.
set_cached_context	Implements set cached context capability.
invalidate_on_change	Implements invalidate on change capability.
warm_common_relationships	Implements warm common relationships capability.

Dependencies: Redis/Memory Cache, RelationshipRepository

Request and Response Examples

Create AI Agent Tool Relationship

```
{
  "source_type": "AI_AGENT",
  "source_id": "AGT-GOV-001",
  "relationship_type": "USES_TOOL",
  "target_type": "TOOL",
  "target_id": "TOOL-REGISTRY-READ",
  "relationship_scope": "WORKFLOW:WF-MODEL-RISK-REVIEW",
  "metadata_json": {
    "access_mode": "READ_ONLY",
    "allowed_operations": [
      "READ_MODEL",
      "READ_AUDIT"
    ],
    "blocked_operations": [
      "UPDATE_POLICY",
      "EXECUTE_EXTERNAL_ACTION"
    ]
  }
}
```

Create Policy Binding Relationship

```
{
  "source_type": "WORKFLOW",
  "source_id": "WF-MODEL-RISK-REVIEW",
  "relationship_type": "GOVERNED_BY",
  "target_type": "POLICY",
  "target_id": "POL-AI-HIGH-RISK-REVIEW",
  "relationship_scope": "TENANT:INNOVANT",
  "metadata_json": {
    "mandatory": true,
    "priority": 10
  }
}
```

Create Evidence Link

```
{
  "source_type": "EVIDENCE",
  "source_id": "EVD-2026-001",
  "relationship_type": "SUPPORTS",
  "target_type": "RECOMMENDATION",
  "target_id": "REC-2026-1044",
  "metadata_json": {
    "evidence_type": "AUDIT_LOG",
    "confidence": 0.91
  }
}
```

Revoke Relationship

```
{
  "relationship_id": "REL-2026-00042",
  "reason": "Tool access no longer approved after security review",
  "effective_to": "2026-06-30T18:00:00+05:30",
  "create_audit_event": true
}
```

Graph Impact Analysis

```
{
  "root_object_type": "AI_MODEL",
  "root_object_id": "MOD-LLM-001",
  "change_type": "RETIREMENT",
  "max_depth": 4,
  "include_inactive": false
}
```

Scenario Playbook - Developer Implementation Examples

Scenario 01: AI Agent Tool Access

Area	Details
Context	An AI agent can read registry data but cannot execute write operations.
Relationships Used	AGT-GOV-001 USES_TOOL TOOL-REGISTRY-READ with access_mode READ_ONLY.
Expected Outcome	Attempted WRITE operation is blocked and AI_ACTION_BLOCKED event is created.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 02: Model Retirement Impact Analysis

Area	Details
Context	An AI model is marked for retirement.
Relationships Used	MOD-LLM-001 is used by 3 agents and 5 workflows.
Expected Outcome	Impact graph lists all affected schedules, workflows, owners and risk items.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 03: Delegated Approval

Area	Details
Context	A risk manager delegates approval for a defined period.
Relationships Used	USR-RISK-001 DELEGATED_TO USR-RISK-002 for HIGH_RISK_APPROVAL until date.
Expected Outcome	Approval is allowed only during date range and audit shows both users.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 04: High-Risk Recommendation

Area	Details
Context	A recommendation has risk score 91.
Relationships Used	REC-001 EVALUATED_BY POL-HIGH-RISK and REQUIRES APPROVAL_GROUP RISK_BOARD.
Expected Outcome	Recommendation cannot become decision until approval is captured.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 05: Evidence-Based Decision

Area	Details
Context	Approval requires evidence.
Relationships Used	REC-001 SUPPORTED_BY EVD-003 and EVD-004.
Expected Outcome	Reviewer sees evidence list before decision.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 06: Policy Binding to Workflow

Area	Details
Context	Workflow must always run under AI governance policy.
Relationships Used	WF-001 GOVERNED_BY POL-AI-GOV-001.
Expected Outcome	Schedule activation blocked if binding inactive.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 07: Cross-Tenant Protection

Area	Details
Context	A relationship attempts to connect objects from different tenants.
Relationships Used	source tenant != target tenant.
Expected Outcome	Relationship creation blocked.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 08: External Auditor Scope

Area	Details
Context	External auditor needs temporary access.
Relationships Used	USR-EXT-001 MEMBER_OF AUDIT_SCOPE with effective_to.
Expected Outcome	Access expires automatically and timeline remains.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 09: Circular Ownership

Area	Details
Context	Department A owns Department B while B owns A.
Relationships Used	Circular OWNED_BY chain.
Expected Outcome	Validation engine blocks relationship.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 10: Broken Graph Reference

Area	Details
Context	Target object no longer exists.
Relationships Used	Relationship points to missing tool.
Expected Outcome	Relationship is invalid; active execution blocked.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 11: Policy Violation Remediation

Area	Details
Context	Agent violates tool boundary.
Relationships Used	AGT-001 requested EXECUTE on READ_ONLY tool.
Expected Outcome	Violation creates remediation action and escalation.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 12: Workflow Schedule Governance

Area	Details
Context	Scheduled run uses agent and model.
Relationships Used	Schedule HAS_ASSIGNMENT AGT-001, USES_MODEL MOD-001.
Expected Outcome	Run records all relationships used at execution time.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 13: Prompt Template Governance

Area	Details
Context	Agent uses governed prompt.
Relationships Used	AGT-001 USES_PROMPT PRM-001 version v3.
Expected Outcome	Recommendation captures prompt version for reproducibility.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 14: Data Source Classification

Area	Details
Context	Data source becomes Restricted.
Relationships Used	DS-001 classification changes to RESTRICTED.
Expected Outcome	Impact analysis identifies agents/tools using it.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 15: Approval Group Change

Area	Details
Context	New approver is added to board.
Relationships Used	USR-010 MEMBER_OF GRP-AI-GOV effective from date.
Expected Outcome	User can approve only after effective_from.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 16: Runtime Boundary Check

Area	Details
Context	Agent starts workflow run.
Relationships Used	Run resolves agent-tool-model relationships before invocation.
Expected Outcome	Run fails fast if boundary invalid.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 17: Evidence Sensitivity

Area	Details
Context	Evidence is Confidential.
Relationships Used	EVD-001 has sensitivity CONFIDENTIAL.
Expected Outcome	Viewer must have matching audit or compliance scope.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 18: Policy Version Upgrade

Area	Details
Context	Policy v2 replaces v1.
Relationships Used	POL-001 has active version v2.
Expected Outcome	Future evaluations use v2; history retains v1.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 19: Risk Escalation

Area	Details
Context	Run output contains critical finding.
Relationships Used	OUTPUT-001 risk CRITICAL.
Expected Outcome	Relationship to escalation owner is resolved and notification sent.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Scenario 20: System Actor Event

Area	Details
Context	Scheduler creates workflow run.
Relationships Used	Actor type SYSTEM creates WORKFLOW_RUN_STARTED.
Expected Outcome	Audit trail shows system actor and schedule id.
Events Required	RELATIONSHIP_RESOLVED, AUTHORIZATION_EVALUATED, GOVERNANCE_CONTEXT_BUILT, domain-specific lifecycle event
QA Evidence	API response, database record, audit timeline screenshot

Expanded Validation Catalogue

Validation Rules 1 to 15

Rule ID	Category	Rule	Severity	Failure Handling
RV-001	Source Object	Validate source object rule 1: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-002	Target Object	Validate target object rule 2: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-003	Relationship Type	Validate relationship type rule 3: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-004	Scope	Validate scope rule 4: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-005	Temporal Validity	Validate temporal validity rule 5: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-006	Lifecycle	Validate lifecycle rule 6: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-007	Authorization	Validate authorization rule 7: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-008	Ownership	Validate ownership rule 8: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-009	Policy	Validate policy rule 9: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-010	Evidence	Validate evidence rule 10: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-011	AI Agent Boundary	Validate ai agent boundary rule 11: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-012	Audit	Validate audit rule 12: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-013	Cross Tenant	Validate cross tenant rule 13: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-014	Duplicate	Validate duplicate rule 14: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-015	Performance	Validate performance rule 15: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.

Validation Rules 16 to 30

Rule ID	Category	Rule	Severity	Failure Handling
RV-016	Source Object	Validate source object rule 16: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-017	Target Object	Validate target object rule 17: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-018	Relationship Type	Validate relationship type rule 18: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-019	Scope	Validate scope rule 19: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-020	Temporal Validity	Validate temporal validity rule 20: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-021	Lifecycle	Validate lifecycle rule 21: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-022	Authorization	Validate authorization rule 22: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-023	Ownership	Validate ownership rule 23: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-024	Policy	Validate policy rule 24: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-025	Evidence	Validate evidence rule 25: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-026	AI Agent Boundary	Validate ai agent boundary rule 26: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-027	Audit	Validate audit rule 27: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-028	Cross Tenant	Validate cross tenant rule 28: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-029	Duplicate	Validate duplicate rule 29: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-030	Performance	Validate performance rule 30: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.

Validation Rules 31 to 45

Rule ID	Category	Rule	Severity	Failure Handling
RV-031	Source Object	Validate source object rule 31: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-032	Target Object	Validate target object rule 32: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-033	Relationship Type	Validate relationship type rule 33: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-034	Scope	Validate scope rule 34: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-035	Temporal Validity	Validate temporal validity rule 35: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-036	Lifecycle	Validate lifecycle rule 36: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-037	Authorization	Validate authorization rule 37: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-038	Ownership	Validate ownership rule 38: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-039	Policy	Validate policy rule 39: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-040	Evidence	Validate evidence rule 40: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-041	AI Agent Boundary	Validate ai agent boundary rule 41: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-042	Audit	Validate audit rule 42: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-043	Cross Tenant	Validate cross tenant rule 43: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-044	Duplicate	Validate duplicate rule 44: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-045	Performance	Validate performance rule 45: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.

Validation Rules 46 to 60

Rule ID	Category	Rule	Severity	Failure Handling
RV-046	Source Object	Validate source object rule 46: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-047	Target Object	Validate target object rule 47: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-048	Relationship Type	Validate relationship type rule 48: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-049	Scope	Validate scope rule 49: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-050	Temporal Validity	Validate temporal validity rule 50: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-051	Lifecycle	Validate lifecycle rule 51: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-052	Authorization	Validate authorization rule 52: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-053	Ownership	Validate ownership rule 53: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-054	Policy	Validate policy rule 54: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-055	Evidence	Validate evidence rule 55: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-056	AI Agent Boundary	Validate ai agent boundary rule 56: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-057	Audit	Validate audit rule 57: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-058	Cross Tenant	Validate cross tenant rule 58: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-059	Duplicate	Validate duplicate rule 59: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-060	Performance	Validate performance rule 60: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.

Validation Rules 61 to 75

Rule ID	Category	Rule	Severity	Failure Handling
RV-061	Source Object	Validate source object rule 61: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-062	Target Object	Validate target object rule 62: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-063	Relationship Type	Validate relationship type rule 63: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-064	Scope	Validate scope rule 64: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-065	Temporal Validity	Validate temporal validity rule 65: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-066	Lifecycle	Validate lifecycle rule 66: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-067	Authorization	Validate authorization rule 67: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-068	Ownership	Validate ownership rule 68: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-069	Policy	Validate policy rule 69: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-070	Evidence	Validate evidence rule 70: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-071	AI Agent Boundary	Validate ai agent boundary rule 71: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-072	Audit	Validate audit rule 72: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-073	Cross Tenant	Validate cross tenant rule 73: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-074	Duplicate	Validate duplicate rule 74: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-075	Performance	Validate performance rule 75: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.

Validation Rules 76 to 90

Rule ID	Category	Rule	Severity	Failure Handling
RV-076	Source Object	Validate source object rule 76: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-077	Target Object	Validate target object rule 77: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-078	Relationship Type	Validate relationship type rule 78: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-079	Scope	Validate scope rule 79: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-080	Temporal Validity	Validate temporal validity rule 80: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-081	Lifecycle	Validate lifecycle rule 81: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-082	Authorization	Validate authorization rule 82: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-083	Ownership	Validate ownership rule 83: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-084	Policy	Validate policy rule 84: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-085	Evidence	Validate evidence rule 85: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-086	AI Agent Boundary	Validate ai agent boundary rule 86: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-087	Audit	Validate audit rule 87: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-088	Cross Tenant	Validate cross tenant rule 88: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-089	Duplicate	Validate duplicate rule 89: relationship must satisfy canonical semantics and operational safety conditions.	MEDIUM	Block transaction and return field-level error; publish validation failure event for sensitive operations.
RV-090	Performance	Validate performance rule 90: relationship must satisfy canonical semantics and operational safety conditions.	HIGH	Block transaction and return field-level error; publish validation failure event for sensitive operations.

Expanded QA Matrix

QA Test Cases 1 to 15

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-001	DB	Execute relationship architecture test scenario 1.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-002	UI	Execute relationship architecture test scenario 2.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-003	Security	Execute relationship architecture test scenario 3.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-004	Audit	Execute relationship architecture test scenario 4.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-005	Graph	Execute relationship architecture test scenario 5.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-006	Performance	Execute relationship architecture test scenario 6.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-007	Negative	Execute relationship architecture test scenario 7.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-008	API	Execute relationship architecture test scenario 8.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-009	DB	Execute relationship architecture test scenario 9.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-010	UI	Execute relationship architecture test scenario 10.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-011	Security	Execute relationship architecture test scenario 11.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-012	Audit	Execute relationship architecture test scenario 12.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-013	Graph	Execute relationship architecture test scenario 13.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-014	Performance	Execute relationship architecture test scenario 14.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-015	Negative	Execute relationship architecture test scenario 15.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

QA Test Cases 16 to 30

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-016	API	Execute relationship architecture test scenario 16.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-017	DB	Execute relationship architecture test scenario 17.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-018	UI	Execute relationship architecture test scenario 18.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-019	Security	Execute relationship architecture test scenario 19.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-020	Audit	Execute relationship architecture test scenario 20.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-021	Graph	Execute relationship architecture test scenario 21.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-022	Performance	Execute relationship architecture test scenario 22.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-023	Negative	Execute relationship architecture test scenario 23.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-024	API	Execute relationship architecture test scenario 24.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-025	DB	Execute relationship architecture test scenario 25.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-026	UI	Execute relationship architecture test scenario 26.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-027	Security	Execute relationship architecture test scenario 27.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-028	Audit	Execute relationship architecture test scenario 28.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-029	Graph	Execute relationship architecture test scenario 29.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-030	Performance	Execute relationship architecture test scenario 30.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

QA Test Cases 31 to 45

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-031	Negative	Execute relationship architecture test scenario 31.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-032	API	Execute relationship architecture test scenario 32.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-033	DB	Execute relationship architecture test scenario 33.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-034	UI	Execute relationship architecture test scenario 34.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-035	Security	Execute relationship architecture test scenario 35.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-036	Audit	Execute relationship architecture test scenario 36.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-037	Graph	Execute relationship architecture test scenario 37.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-038	Performance	Execute relationship architecture test scenario 38.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-039	Negative	Execute relationship architecture test scenario 39.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-040	API	Execute relationship architecture test scenario 40.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-041	DB	Execute relationship architecture test scenario 41.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-042	UI	Execute relationship architecture test scenario 42.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-043	Security	Execute relationship architecture test scenario 43.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-044	Audit	Execute relationship architecture test scenario 44.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-045	Graph	Execute relationship architecture test scenario 45.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

QA Test Cases 46 to 60

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-046	Performance	Execute relationship architecture test scenario 46.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-047	Negative	Execute relationship architecture test scenario 47.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-048	API	Execute relationship architecture test scenario 48.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-049	DB	Execute relationship architecture test scenario 49.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-050	UI	Execute relationship architecture test scenario 50.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-051	Security	Execute relationship architecture test scenario 51.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-052	Audit	Execute relationship architecture test scenario 52.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-053	Graph	Execute relationship architecture test scenario 53.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-054	Performance	Execute relationship architecture test scenario 54.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-055	Negative	Execute relationship architecture test scenario 55.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-056	API	Execute relationship architecture test scenario 56.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-057	DB	Execute relationship architecture test scenario 57.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-058	UI	Execute relationship architecture test scenario 58.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-059	Security	Execute relationship architecture test scenario 59.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-060	Audit	Execute relationship architecture test scenario 60.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

QA Test Cases 61 to 75

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-061	Graph	Execute relationship architecture test scenario 61.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-062	Performance	Execute relationship architecture test scenario 62.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-063	Negative	Execute relationship architecture test scenario 63.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-064	API	Execute relationship architecture test scenario 64.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-065	DB	Execute relationship architecture test scenario 65.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-066	UI	Execute relationship architecture test scenario 66.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-067	Security	Execute relationship architecture test scenario 67.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-068	Audit	Execute relationship architecture test scenario 68.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-069	Graph	Execute relationship architecture test scenario 69.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-070	Performance	Execute relationship architecture test scenario 70.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-071	Negative	Execute relationship architecture test scenario 71.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-072	API	Execute relationship architecture test scenario 72.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-073	DB	Execute relationship architecture test scenario 73.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-074	UI	Execute relationship architecture test scenario 74.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-075	Security	Execute relationship architecture test scenario 75.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

QA Test Cases 76 to 90

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-076	Audit	Execute relationship architecture test scenario 76.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-077	Graph	Execute relationship architecture test scenario 77.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-078	Performance	Execute relationship architecture test scenario 78.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-079	Negative	Execute relationship architecture test scenario 79.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-080	API	Execute relationship architecture test scenario 80.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-081	DB	Execute relationship architecture test scenario 81.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-082	UI	Execute relationship architecture test scenario 82.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-083	Security	Execute relationship architecture test scenario 83.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-084	Audit	Execute relationship architecture test scenario 84.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-085	Graph	Execute relationship architecture test scenario 85.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-086	Performance	Execute relationship architecture test scenario 86.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-087	Negative	Execute relationship architecture test scenario 87.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-088	API	Execute relationship architecture test scenario 88.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-089	DB	Execute relationship architecture test scenario 89.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-090	UI	Execute relationship architecture test scenario 90.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

QA Test Cases 91 to 100

Test ID	Area	Scenario	Expected Result	Status	Evidence
QA-091	Security	Execute relationship architecture test scenario 91.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-092	Audit	Execute relationship architecture test scenario 92.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-093	Graph	Execute relationship architecture test scenario 93.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-094	Performance	Execute relationship architecture test scenario 94.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-095	Negative	Execute relationship architecture test scenario 95.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-096	API	Execute relationship architecture test scenario 96.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-097	DB	Execute relationship architecture test scenario 97.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-098	UI	Execute relationship architecture test scenario 98.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-099	Security	Execute relationship architecture test scenario 99.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence
QA-100	Audit	Execute relationship architecture test scenario 100.	Expected result must match lifecycle, validation and audit rules.	Pass/Fail	Screenshot/API/database evidence

Developer Implementation Checklist

#	Checklist Item	Priority	Notes
1	Database migrations are reversible	Required	Owner to update during implementation
2	All tables include tenant_id	Required	Owner to update during implementation
3	All active queries filter is_deleted/status	Required	Owner to update during implementation
4	Composite indexes created	Required	Owner to update during implementation
5	SQLAlchemy models match DB	Required	Owner to update during implementation
6	Pydantic schemas validate enums	Required	Owner to update during implementation
7	All write APIs call authorization	Required	Owner to update during implementation
8	All write APIs publish audit event	Required	Owner to update during implementation
9	Validation engine covers mandatory rules	Required	Owner to update during implementation
10	Generic relationship and specific mappings both supported	Required	Owner to update during implementation
11	Graph traversal has depth limits	Required	Owner to update during implementation
12	Graph output hides unauthorized objects	Required	Owner to update during implementation
13	Cache invalidates on relationship update	Required	Owner to update during implementation
14	No hard delete for production relationships	Required	Owner to update during implementation
15	Error messages are user-friendly	Required	Owner to update during implementation
16	OpenAPI examples are complete	Required	Owner to update during implementation
17	Unit tests created	Required	Owner to update during implementation
18	Integration tests created	Required	Owner to update during implementation
19	Negative tests created	Required	Owner to update during implementation
20	Performance tests executed	Required	Owner to update during implementation
21	UAT scenarios mapped	Required	Owner to update during implementation
22	Documentation updated	Required	Owner to update during implementation
23	PM sign-off captured	Required	Owner to update during implementation